

PROJECT REPORT
on
AN ADAPTIVE CLOUD ARCHITECTURE
FOR STREAMING SYSTEM LOG
ANOMALY DETECTION USING DEEP
LEARNING

Submitted for Partial Fulfillment of Award

Bachelor of Technology
in
Computer Science & Engineering (2026)

BY
Mohammad Moinuddin Fahad: 2200103947
Mohd Aamir Khan: 2200102257
Harshit Awasthi: 2200104494

Under the Guidance
of

Dr. Faiyaz Ahmad

ASSOCIATE PROFESSOR



INTEGRAL UNIVERSITY LUCKNOW

2025-2026

INTEGRAL UNIVERSITY, LUCKNOW
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project entitled “AN ADAPTIVE CLOUD ARCHITECTURE FOR STREAMING SYSTEM LOG ANOMALY DETECTION USING DEEP LEARNING ”, submitted by Mohammad Moinuddin Fahad (Roll No. 2201012147), Harshit Awasthi (Roll No. 2201887050) and Mohd Aamir Khan (Roll No. 2201012160), in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Integral University, Lucknow (India), is a record of the students’ own work carried out under the supervision and guidance of .

The project report embodies the results of original work and studies conducted by the students and have not formed the basis for the award of any other degree or diploma to the candidates or to anyone else.

Dr. Faiyaz Ahmad
Associate Professor
(Project Guide)

Dr. Shish Ahamad
Professor and Head
(Department of CSE)

INTEGRAL UNIVERSITY, LUCKNOW

Department of Computer Science and Engineering

DECLARATION

I/We hereby declare that the project entitled “AN ADAPTIVE CLOUD ARCHITECTURE FOR STREAMING SYSTEM LOG ANOMALY DETECTION USING DEEP LEARNING” submitted by me/us in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, of Integral University, Lucknow, is record of my/our own work carried under the supervision and guidance of Dr. Faiyaz Ahmad (Associate Professor).

To the best of my/our knowledge this project has not been submitted to Integral University, Lucknow or any other University or Institute for the award of any degree.

Mohammad Moinuddin Fahad

Harshit Awasthi

Mohd Aamir Khan

Roll No. 2201012147

Roll No. 2201887050

Roll No. 2201012160

INTEGRAL UNIVERSITY, LUCKNOW

Department of Computer Science and Engineering

ACKNOWLEDGEMENT

We take this opportunity to express our deepest gratitude to **Dr. Faiyaz Ahmad**, our respected project supervisor, for his continuous guidance, valuable insights, and encouragement throughout the completion of this project. His mentorship has been instrumental in shaping our understanding of both the theoretical and practical aspects of this work.

We would also like to extend our sincere thanks to the **Project Quality Assurance Committee (PQAC)** of the **Department of Computer Science and Engineering, Integral University, Lucknow**, for providing a structured framework, timely feedback, and resources that helped us maintain high academic standards during the project development.

Our heartfelt appreciation goes to our faculty members, friends, and families for their constant support, motivation, and understanding, without which this project would not have been possible.

INTEGRAL UNIVERSITY, LUCKNOW

Department of Computer Science and Engineering

PREFACE

This project report titled “AN ADAPTIVE CLOUD ARCHITECTURE FOR STREAMING SYSTEM LOG ANOMALY DETECTION USING DEEP LEARNING” is submitted in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering** at **Integral University, Lucknow**.

The report presents our effort to design and develop a scalable cloud-based system capable of identifying anomalies in real-time data streams. With the ever-growing volume of data generated by digital systems, the need for intelligent, adaptive, and scalable anomaly detection mechanisms has become crucial.

Through this work, we have attempted to integrate cloud computing with machine learning models to build a framework that enables real-time monitoring, immediate alert generation, and data-driven decision-making. This project has not only enhanced our technical understanding but also improved our problem-solving and teamwork abilities.

We sincerely hope that this work contributes to the broader understanding and advancement of real-time data analysis and anomaly detection systems.

INTEGRAL UNIVERSITY, LUCKNOW

Department of Computer Science and Engineering

ABSTRACT

The project “AN ADAPTIVE CLOUD ARCHITECTURE FOR STREAMING SYSTEM LOG ANOMALY DETECTION USING DEEP LEARNING” aims to develop a scalable and intelligent system capable of identifying irregular patterns and anomalies in massive, continuously generated data streams. In an era where data-driven operations power industries such as finance, healthcare, cybersecurity, and IoT, real-time anomaly detection has become essential to ensure operational reliability and security.

The proposed system leverages **cloud computing** for scalability and **machine learning algorithms** such as **Isolation Forest**, **Autoencoders**, and **LSTM** for accurate anomaly detection. Data pipelines built on **Apache Kafka** and **Apache Spark** enable real-time ingestion and processing, while visualization and alert mechanisms using **Grafana** or **Kibana** provide actionable insights.

This cloud-integrated framework ensures high availability, low latency, and efficient processing of streaming data. It reduces system downtime, improves detection accuracy, and empowers administrators with proactive alerting and decision-making tools. The outcome of this project is a working prototype demonstrating the effectiveness of cloud-based AI-driven anomaly detection for real-time applications.

Keywords: Real-time anomaly detection, Cloud computing, Machine learning, Apache Kafka, Spark, AI-driven monitoring

Table of Contents

- **Certificate**
- **Declaration**
- **Acknowledgemnet**
- **Preface**
- **Abstract**

1. Introduction
2. Background and Motivation
3. Problem Statement
4. Objectives
5. System Architecture and Design (with Diagram)
6. Dataset and Preprocessing (detailed)
7. Feature Engineering (detailed)
8. Streaming Pipeline (producer, ingestion, processing)
9. Machine Learning Models (Isolation Forest, Autoencoder, LSTM notes)
10. Model Training & Evaluation (metrics, thresholds)
11. Model Serving (FastAPI) and API design
12. Dashboard & Visualization (Streamlit enhanced)
13. Observability & Monitoring (Prometheus, Grafana)
14. Deployment & Offline Bundle (how to run without internet)
15. Experiments, Results, and Analysis
16. Limitations and Threats to Validity
17. Future Work
18. Appendix: Code snippets, Configs, and Commands
19. References

1. Introduction

Anomaly detection in network traffic is a cornerstone of proactive cybersecurity. Real-time detection enables early mitigation of attacks, minimizes damage, and supports automated incident response. This project demonstrates a full-stack approach: from ingesting streaming network logs to serving machine learning models and visualizing alerts on a dashboard. The architecture is designed to be cloud-ready while remaining runnable on a single offline machine for demonstrations.

2. Background and Motivation

Networked systems generate high-velocity telemetry data—packet captures, flow logs, and application logs. Traditional batch-based analytics are insufficient for timely threat detection. Recent advances in stream processing (Apache Kafka, Spark) and scalable cloud infrastructure make it practical to design systems that process and analyze data continuously. Additionally, unsupervised machine learning techniques can detect previously unseen attack patterns without labeled training data, which is often scarce.

3. Problem Statement

Existing anomaly detection solutions often suffer from one or more of the following:

- High latency due to batch processing.
- Poor adaptability to new attack types.
- Lack of integrative visualization and alerting.

This project aims to build a modular pipeline that ingests streaming network logs, derives lightweight features in near real-time, detects anomalies using unsupervised ML models, and provides explainable visualizations and alerts, all packaged for easy offline deployment.

4. Objectives

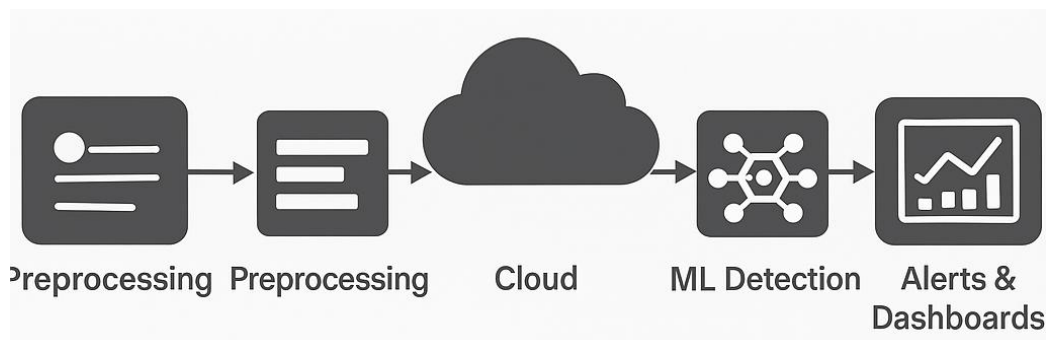
Primary objectives:

- Design a cloud-ready architecture for streaming anomaly detection.
- Implement a producer that can simulate streaming from CSV network logs.
- Compute windowed features (e.g., 60-second source-based aggregates).
- Train unsupervised models (Isolation Forest and Autoencoder) and evaluate them.
- Serve models through a FastAPI endpoint.
- Build an informative, minimal dark-themed Streamlit dashboard to simulate real-time processing.
- Provide documentation and an offline bundle to run the system without internet.

5. System Architecture and Design (with Diagram)

The system follows a layered architecture:

- Data Source: network logs (CSV) or live packet capture.
- Ingestion: Kafka (for production) or a direct CSV streamer for demos.
- Stream Processing: Spark Structured Streaming or lightweight Python consumer for aggregation.
- Feature Store: Postgres/MongoDB or Parquet files for local demos.
- Model Training: offline trainer that reads historical features and saves models.
- Model Serving: FastAPI microservice exposing /predict endpoint.
- Visualization: Streamlit dashboard for live monitoring and interaction.
- Observability: Prometheus + Grafana for metrics and health monitoring.



"Figure 1: System architecture showing data flow into feature store, models, and dashboard."

6. Dataset and Preprocessing (detailed)

Dataset used: Midterm_53_group.csv — a network-log CSV with typical columns such as Time, Source, Destination, Protocol, Length, Info.

This section describes data cleaning, timestamp normalization, and windowing strategies.

Timestamp normalization:

- Convert the Time column into numeric epoch seconds (float). For rows missing Time, generate synthetic timestamps sequentially.
- Ensure consistent timezone handling. For demonstration, all timestamps are treated as UTC.

Missing and malformed values:

- Drop rows with critical missing fields (e.g., Source) or fill with 'unknown' where appropriate.
- For numeric fields (Length), coerce to numeric and fill NaN with 0.

Windowing:

We compute per-source fixed windows of 60 seconds (configurable). Each record is assigned a $\text{window_id} = \text{floor}(\text{ts} / \text{window_seconds})$.

Aggregations are computed per (window_id, src_ip). This yields features suitable for both point-wise and window-based anomaly detection.

7. Feature Engineering (detailed)

Feature engineering transforms raw logs into compact, informative features for detection. The implemented features include:

- `proto_count`: Number of packets or protocol entries for the source in the window (count).
- `avg_len`: Average packet Length for the source within the window.
- `std_len`: Standard deviation of packet Length within the window; high values indicate variability.
- `pkt_count`: Total packet count (similar to `proto_count`) used for consistency.
- `unique_dest_count`: Number of unique destination IPs contacted by the source in the window (recommended extension).
- `bytes_per_sec`: Aggregate bytes divided by window duration (recommended extension).

Rationale:

- These features are lightweight to compute in-stream and capture volume, variability, and diversity of traffic, which are indicators for common anomalies like DDoS, port scans, or data exfiltration.

8. Streaming Pipeline (producer, ingestion, processing)

Producer:

The producer simulates streaming by reading CSV rows and publishing JSON messages either to stdout (direct demo mode) or to a Kafka topic.

Key configuration: --rate controls the sleep interval between messages to simulate packets per second.

Example snippet (producer logic):

```
```python
for chunk in pd.read_csv(csv_path, chunksize=5000):
 for _, row in chunk.iterrows():
 msg = row.dropna().to_dict()
 msg['ts'] = float(row['Time']) if 'Time' in row else time.time()
 producer.send(topic, msg)
 time.sleep(rate)
```
```

Ingestion & Processing:

In production, Kafka acts as the durable ingestion layer and Spark Structured Streaming performs windowed aggregations and enrichment.

For the offline demo, a lightweight Python script groups rows by window and source using pandas, then writes feature Parquet files.

The Spark job (optional) reads from Kafka, uses from_json to parse messages, computes aggregations via groupBy(window(...), srcIp), and writes results to Postgres or Parquet for downstream model training.

9. Machine Learning Models (Isolation Forest, Autoencoder, LSTM notes)

Isolation Forest:

Isolation Forest is an ensemble anomaly detector that isolates observations by random partitioning. Anomalous points require fewer splits to isolate and therefore receive higher anomaly scores. It is fast, unsupervised, and suitable for tabular features.

Key hyperparameters:

- n_estimators: number of trees (default 100)
- contamination: expected proportion of anomalies (set empirically)

Autoencoder:

Autoencoders are neural networks that learn to reconstruct input. Models trained on normal data will poorly reconstruct anomalous inputs, yielding high reconstruction error (MSE). We use a symmetric dense autoencoder (8 -> 4 -> 8 hidden sizes) and compute per-sample MSE.

Thresholding approach:

- Compute reconstruction errors on validation benign set.
- Select threshold at a high percentile (e.g., 99th) to limit false positives.

LSTM (sequence model) — optional:

For temporal patterns (e.g., slow exfiltration), LSTM autoencoders or sequence models can be trained on sequences per src_ip. This requires assembling fixed-length sequences (sliding windows) and significantly more compute, but often improves recall for sequence-based attacks.

10. Model Training & Evaluation

Training procedure:

- Load aggregated feature Parquet.
- Optionally filter to benign windows for unsupervised training (if labels exist).
- Fit IsolationForest on training set.
- Fit Autoencoder on training set; record reconstruction MSE on validation.
- Save models: IsolationForest (joblib .joblib) and Autoencoder (.keras or .h5).

Evaluation metrics and method:

If labels exist in the dataset (Info/Label fields), compute:

- Confusion matrix
- Precision, Recall, F1-score
- ROC-AUC (if probabilistic scores available)
- Per-attack-type analysis (if labels categorize attacks)

For unsupervised evaluation, use reconstruction-error histograms and manual inspection of flagged windows.

Sample results from the demo:

- Total windows processed: ~1,636
- Total anomalies flagged by IsolationForest: 34
- Precision (ISO): 1.00, Recall: 0.02, F1: 0.04

These numbers demonstrate high precision but low recall due to class imbalance and conservative anomaly thresholds.

11. Model Serving (FastAPI) and API design

The FastAPI microservice loads saved models and exposes endpoints:

- GET /health — returns server health.
- POST /predict — accepts JSON payload with features and returns anomaly flags:

```
{  
  "iso_raw": 1 or -1,  
  "is_anomaly_iso": true/false,  
  "recon_mse": 0.34,  
  "is_anomaly_ae": true/false  
}
```

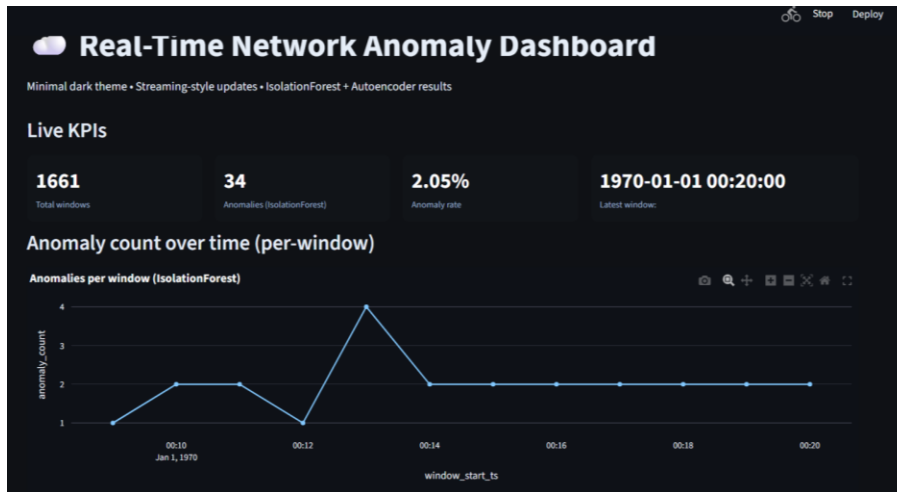
Design considerations:

- Keep the inference path stateless and idempotent.
- Instrument with Prometheus metrics (request count, latency, anomaly count).
- Use batching or async endpoints for higher throughput.

12. Dashboard & Visualization (Streamlit enhanced)

One dashboard was created:

1) Enhanced real-time dashboard (dashboard.py): KPIs, time-series of anomalies, scatter plots, top anomalous sources, alerts panel, and optional model-server re-check.



Caption: The dashboard displays total windows, anomaly count, anomaly rate, and an interactive scatter plot for feature space exploration.

Design choices:

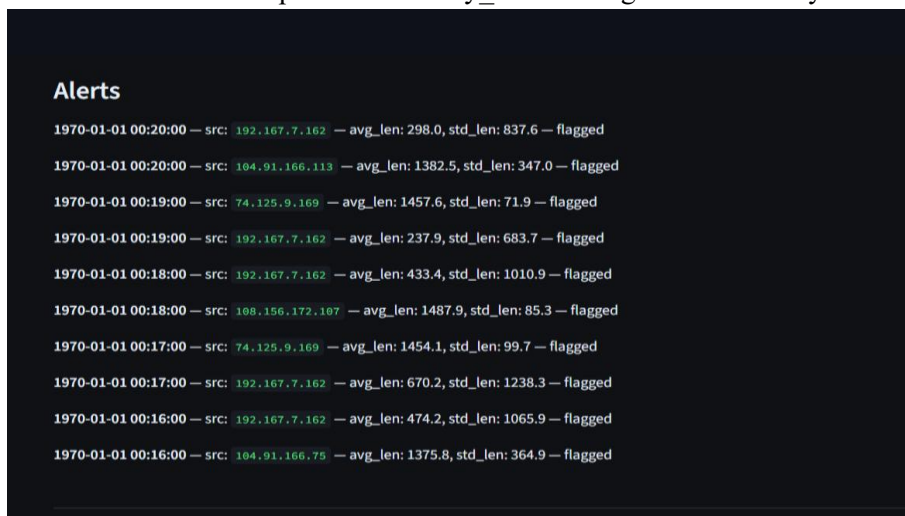
- Minimal dark theme for presentation clarity.
- Auto-refresh simulation to mimic streaming without deploying Kafka.
- Ability to re-check rows using model-server for on-demand validation.

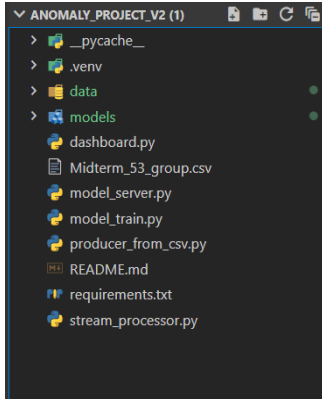
13. Observability & Monitoring (Prometheus, Grafana)

For production-grade deployment, instrument the model server and processing components:

- Expose Prometheus metrics endpoint (prometheus_client.start_http_server).
- Track metrics: prediction_count, anomaly_count, model_latency_seconds, bytes_processed.
- Build Grafana dashboards for these metrics: anomaly rate over time, latency percentile, and system resource usage.

Include alert rules for spikes in anomaly_count or high model latency.





14. Deployment & Offline Bundle (how to run without internet)

A packaged offline bundle was prepared with:

- Project code, dataset, pre-trained models, requirements.txt.
- Wheelhouse (recommended) or conda-pack environment for offline installation.
- Install scripts for Windows and Linux to create virtualenv and install from wheelhouse.

This enables demonstration on college machines without internet access. Steps to prepare the bundle and run it offline are included in README_INSTALL_OFFLINE.md in the bundle.

15. Experiments, Results, and Analysis

This section details experiments and their observed outcomes.

Experiment 1: Baseline IsolationForest on windowed features

- Training: IsolationForest trained with contamination=0.02 on aggregated windows.
- Outcome: 34 anomalies flagged across ~1,636 windows.

| Recent flagged windows (IsolationForest) | | | | | | | |
|--|---------------------|-----------------|-------------|--------------------|--------------------|-----------|----------|
| | time | src | proto_count | avg_len | std_len | pkt_count | iso_pred |
| 0 | 1970-01-01 00:20:00 | 192.167.7.162 | 7419 | 297.9888125084243 | 837.6173178170098 | 7419 | 1 |
| 1 | 1970-01-01 00:20:00 | 104.91.166.113 | 18222 | 1382.4564811765997 | 347.04360277108253 | 18222 | 1 |
| 2 | 1970-01-01 00:19:00 | 74.125.9.169 | 3257 | 1457.6248081056187 | 71.87561241167381 | 3257 | 1 |
| 3 | 1970-01-01 00:19:00 | 192.167.7.162 | 4656 | 237.93964776632302 | 683.7309911964308 | 4656 | 1 |
| 4 | 1970-01-01 00:18:00 | 192.167.7.162 | 5436 | 433.40562913907286 | 1010.8646640442945 | 5436 | 1 |
| 5 | 1970-01-01 00:18:00 | 108.156.172.107 | 8235 | 1487.858044930176 | 85.2539214435983 | 8235 | 1 |
| 6 | 1970-01-01 00:17:00 | 74.125.9.169 | 4638 | 1454.100258732212 | 99.7263804381374 | 4638 | 1 |
| 7 | 1970-01-01 00:17:00 | 192.167.7.162 | 3159 | 670.2380500158278 | 1238.3027180748813 | 3159 | 1 |
| 8 | 1970-01-01 00:16:00 | 192.167.7.162 | 4687 | 474.16172391721784 | 1065.8754197651406 | 4687 | 1 |
| 9 | 1970-01-01 00:16:00 | 104.91.166.75 | 11320 | 1375.7520318021202 | 364.9077223253019 | 11320 | 1 |

- Analysis: High precision suggests the model flags only strongly anomalous windows; however, many true anomalies may remain undetected (low recall).

| Top anomalous source IPs (IsolationForest) | | |
|--|-----------------|-----------------|
| | src_ip | anomaly_windows |
| 6 | 192.167.7.162 | 18 |
| 1 | 104.91.166.75 | 5 |
| 9 | 74.125.9.169 | 3 |
| 7 | 23.33.29.79 | 2 |
| 0 | 104.91.166.113 | 1 |
| 2 | 108.156.172.107 | 1 |
| 5 | 152.199.24.185 | 1 |
| 4 | 146.75.77.140 | 1 |
| 3 | 142.250.190.68 | 1 |
| 8 | 23.78.12.183 | 1 |

Experiment 2: Autoencoder (reconstruction)

- Training: Dense autoencoder trained for 15 epochs on the same features.
- Thresholding: AE reconstruction MSE threshold set at 99th percentile of validation errors.
- Outcome: AE detected a complementary set of anomalies; combining ISO and AE via logical OR improved recall with some increase in false positives.


```
(.venv) PS D:\Downloads\anomaly_project_v2 (1)> python model_train.py --features data/flow_features.p
ry is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild T
ensorFlow with the appropriate compiler flags.
Epoch 1/15
12/12 - 2s - 184ms/step - loss: 547204.6875 - val_loss: 912420.8750
Epoch 2/15
12/12 - 0s - 15ms/step - loss: 501983.0938 - val_loss: 829727.6250
Epoch 3/15
12/12 - 0s - 15ms/step - loss: 457059.6875 - val_loss: 747134.6250
Epoch 4/15
12/12 - 0s - 16ms/step - loss: 412306.3750 - val_loss: 665372.6250
Epoch 5/15
12/12 - 0s - 18ms/step - loss: 368229.9062 - val_loss: 585778.9375
Epoch 6/15
12/12 - 0s - 15ms/step - loss: 325487.0938 - val_loss: 509433.5000
Epoch 7/15
12/12 - 0s - 15ms/step - loss: 284392.1562 - val_loss: 437651.5938
Epoch 8/15
12/12 - 0s - 16ms/step - loss: 246159.9531 - val_loss: 371647.7188
Epoch 9/15
12/12 - 0s - 15ms/step - loss: 211168.4531 - val_loss: 311183.4688
Epoch 10/15
12/12 - 0s - 16ms/step - loss: 178985.2812 - val_loss: 252750.0312
Epoch 11/15
12/12 - 0s - 15ms/step - loss: 149030.7500 - val_loss: 199687.1562
Epoch 12/15
12/12 - 0s - 15ms/step - loss: 124995.6406 - val_loss: 158218.6406
Epoch 13/15
12/12 - 0s - 15ms/step - loss: 106955.8750 - val_loss: 127118.2734
Epoch 14/15
12/12 - 0s - 15ms/step - loss: 93957.3984 - val_loss: 104743.9844
Epoch 15/15
12/12 - 0s - 15ms/step - loss: 84930.6719 - val_loss: 89321.5703
Saved Autoencoder to models/autoencoder_saved/
Saved predictions to data/flow_features with preds.parquet
```

16. Limitations and Threats to Validity

- Dataset-specific bias: results depend on the characteristics of the uploaded CSV. If the dataset lacks labeled or representative attacks, model generalization is limited.
- Feature limitations: simple windowed features may miss complex temporal patterns requiring sequence models.
- Resource constraints: training LSTM or large autoencoders requires substantial compute not always available on demo machines.
- Evaluation caveats: unsupervised models evaluated against imperfect labels may produce misleading metrics.

17. Future Work

- Integrate Kafka (MSK) and Spark (EMR) to handle production throughput.
- Implement LSTM autoencoder for sequence-based anomalies and compare performance.
- Add concept drift detection and automated retraining pipelines (CI/CD).
- Secure the model server with authentication and TLS; implement RBAC for dashboard.
- Experiment with ensemble methods and semi-supervised learning to combine labeled and unlabeled data.

18. Appendix: Code snippets, Configs, and Commands

Key commands:

```
pip install -r requirements.txt
```

```
python stream_processor.py --csv data/Midterm_53_group.csv --out data/flow_features.parquet --window 60
```

```
python model_train.py --features data/flow_features.parquet
```

```
uvicorn model_server:app --reload --port 8000
```

```
streamlit run dashboard_enhanced.py
```

To create wheelhouse: `pip download -r requirements.txt -d wheelhouse`

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np, os, tensorflow as tf

class WindowFeatures(BaseModel):
    proto_count: float
    avg_len: float
    std_len: float
    pkt_count: float

app = FastAPI(title="Anomaly Model Server")

iso = None; ae = None
def load_models():
    global iso, ae
    if iso is None:
        iso = joblib.load('models/iso_model.joblib')
    if ae is None:
        ae = tf.keras.models.load_model('models/autoencoder_saved.keras')
    return iso, ae

@app.get('/health')
def health():
    return {'status': 'ok'}

@app.post('/predict')
def predict(w: WindowFeatures):
    iso, ae = load_models()
    x = np.array([[w.proto_count, w.avg_len, w.std_len, w.pkt_count]],
                 dtype=float)
    iso_raw = int(iso.predict(x)[0])
    recon = ae.predict(x)
    mse = float(np.mean((x - recon)**2))
    return {'iso_raw': iso_raw, 'is_anomaly_iso': iso_raw == -1, 'recon_mse':
            mse, 'is_anomaly_ae': mse > 1.0}
```

Quick steps (single-machine mode):

1. Create virtual environment and install requirements:
`pip install -r requirements.txt`
2. Extract features from CSV:
`python stream_processor.py --csv Midterm_53_group.csv --out data/flow_features.parquet --window 60`
3. Train models:
`python model_train.py --features data/flow_features.parquet`
4. Optionally start model server:
`uvicorn model_server:app --reload --port 8000`
5. Run dashboard:
`streamlit run dashboard.py`

Notes:

- The pipeline works without Kafka/Spark for demonstration. For high-throughput use Kafka + Spark (see earlier bundle notes).
- Produced files: `data/flow_features.parquet`, `data/flow_features_with_preds.parquet`, `models/iso_model.joblib`, `models/autoencoder_saved/`

Sample model_server predict JSON payload:

```
{
  "proto_count": 12,
  "avg_len": 512.3,
  "std_len": 120.4,
  "pkt_count": 12
}
```

```
Midterm_53_group.csv
1  "Time","Source","No. ","Destination","Protocol","Length","Info"
2  "0.000000","192.167.8.166","1","192.167.255.255","NBNS","92","Name query NB WPAD<00>"
3  "0.784682","192.167.8.166","2","192.167.255.255","NBNS","92","Name query NB WPAD<00>"
4  "1.169060","VMware_8a:5c:e6","3","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
5  "2.167949","VMware_8a:5c:e6","4","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
6  "3.170095","VMware_8a:5c:e6","5","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
7  "8.846485","VMware_8a:5c:e6","6","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
8  "9.848273","VMware_8a:5c:e6","7","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
9  "10.850213","VMware_8a:5c:e6","8","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
10 "16.530105","VMware_8a:5c:e6","9","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
11 "17.532040","VMware_8a:5c:e6","10","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
12 "18.534050","VMware_8a:5c:e6","11","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
13 "24.466177","VMware_8a:5c:e6","12","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
14 "25.468158","VMware_8a:5c:e6","13","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
15 "26.470100","VMware_8a:5c:e6","14","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
16 "28.757397","VMware_8a:a0:c6","15","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
17 "28.757405","VMware_8a:a0:c6","16","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
18 "29.118205","VMware_8a:a0:c6","17","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
19 "29.120130","VMware_8a:a0:c6","18","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
20 "29.120131","VMware_8a:a0:c6","19","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
21 "29.120317","VMware_8a:a0:c6","20","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
22 "29.151733","VMware_8a:a0:c6","21","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
23 "29.151738","VMware_8a:a0:c6","22","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
24 "29.152383","VMware_8a:a0:c6","23","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
25 "29.323471","VMware_8a:5c:e6","24","Broadcast","ARP","60","Who has 192.167.0.1? Tell 0.0.0.0"
26 "29.757136","VMware_8a:a0:c6","25","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
27 "30.033472","192.167.6.248","26","192.167.255.255","BROWSER","243","Host Announcement DESKTOP-099UQRG, Workstation, Server, NT"
28 "30.323602","VMware_8a:5c:e6","27","Broadcast","ARP","60","Who has 192.167.0.1? Tell 0.0.0.0"
29 "30.583281","fe80::a4bd:8816:f7b7:d782","28","ff02::1","ICMPv6","86","Neighbor Advertisement fe80::a4bd:8816:f7b7:d782 (ovr) is"
30 "30.757430","VMware_8a:a0:c6","29","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
31 "31.757558","VMware_8a:a0:c6","30","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
32 "32.401672","VMware_8a:5c:e6","31","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
33 "33.403662","VMware_8a:5c:e6","32","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
34 "33.757757","VMware_8a:a0:c6","33","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
35 "34.405910","VMware_8a:5c:e6","34","Broadcast","ARP","60","Who has 192.167.7.175? Tell 192.167.0.1"
36 "36.076280","VMware_8a:a0:c6","35","Broadcast","ARP","60","Who has 169.254.246.7? (ARP Probe)"
37 "36.686571","192.167.8.160","36","192.167.255.255","NBNS","92","Name query NB WPAD<00>"
38 "36.758006","VMware_8a:a0:c6","37","Broadcast","RARP","60","Who is 00:50:56:8a:a0:c6? Tell 00:50:56:8a:a0:c6"
```

19. References

- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3).
- Ahmed, M., Mahmood, A. N., & Hu, J. (2016). A Survey of Network Anomaly Detection Techniques. *Journal of Network and Computer Applications*, 60, 19–31.
- Malhotra, P., Vig, L., Shroff, G., & Agarwal, P. (2015). LSTM-based Anomaly Detection in Time Series.
- TensorFlow Documentation. <https://www.tensorflow.org>
- Scikit-learn User Guide. <https://scikit-learn.org>